TransformerEncoderLayer#

https://pytorch.org/docs/stable/generated/torch.nn.modules.transformer.Trans

Description:

TransformerEncoderLayer is made up of self-attn and feedforward network. This TransformerEncoderLayer implements the original architecture described in the Attention Is All You Need paper. The intent of this layer is as a reference implementation for foundational understanding and thus it contains only limited features relative to newer Transformer architectures. Given the fast pace of innovation in transformer-like architectures, we recommend exploring this tutorial to build efficient layers from building blocks in core or using higher level libraries from the PyTorch Ecosystem. TransformerEncoderLayer can handle either traditional torch.tensor inputs, or Nested Tensor inputs. Derived classes are expected to similarly accept both input formats. (Not all combinations of inputs are currently supported by TransformerEncoderLayer while Nested Tensor is in prototype state.) If you are implementing a custom layer, you may derive it either from the Module or TransformerEncoderLayer class. If your custom layer supports both torch.Tensors and Nested Tensors inputs, make its implementation a derived class of TransformerEncoderLayer. If your custom Layer supports only torch.Tensor inpu

Function Signature

```
class
torch.nn.modules.transformer.TransformerEncoderLayer(d_model
, nhead, dim_feedforward=2048, dropout=0.1, activation=
<function relu>, layer_norm_eps=1e-05, batch_first=False,
norm_first=False, bias=True, device=None, dtype=None)
[source]#

Parameters

Alternatively, when batch_first is True:

Fast path:

forward(src, src_mask=None, src_key_padding_mask=None,
is_causal=False)[source]#
```

Returns:

Tensor

Code Examples

```
>>> encoder_layer = nn.TransformerEncoderLayer(d_model=512,
nhead=8)
>>> src = torch.rand(10, 32, 512)
>>> out = encoder_layer(src)
```

```
>>> encoder_layer = nn.TransformerEncoderLayer(
...     d_model=512, nhead=8, batch_first=True
... )
>>> src = torch.rand(32, 10, 512)
>>> out = encoder_layer(src)
```

Detailed Documentation

Documentation

TransformerEncoderLayer is made up of self-attn and feedforward network.

This TransformerEncoderLayer implements the original architecture described in the Attention Is All You Need paper. The intent of this layer is as a reference implementation for foundational understanding and thus it contains only limited features relative to newer Transformer architectures. Given the fast pace of innovation in transformer-like architectures, we recommend exploring this tutorial to build efficient layers from building blocks in core or using higher level libraries from the PyTorch Ecosystem.

TransformerEncoderLayer can handle either traditional torch.tensor inputs, or Nested Tensor inputs. Derived classes are expected to similarly accept both input formats. (Not all combinations of inputs are currently supported by TransformerEncoderLayer while Nested Tensor is in prototype state.)

If you are implementing a custom layer, you may derive it either from the Module or TransformerEncoderLayer class. If your custom layer supports both torch.Tensors and Nested Tensors inputs, make its implementation a derived class of TransformerEncoderLayer. If your custom Layer supports only torch.Tensor inputs, derive its implementation from Module.

d_model (int) – the number of expected features in the input (required).

nhead (int) – the number of heads in the multiheadattention models (required).

dim_feedforward (int) – the dimension of the feedforward network model (default=2048).

dropout (float) – the dropout value (default=0.1).

activation (Union[str, Callable[[Tensor], Tensor]]) – the activation function of the intermediate layer, can be a string (“relu” or “gelu”) or a unary callable. Default: relu

layer_norm_eps (float) – the eps value in layer normalization components (default=1e-5).

batch_first (bool) – If True, then the input and output tensors are provided as (batch, seq, feature). Default: False (seq, batch, feature).

norm_first (bool) – if True, layer norm is done prior to attention and feedforward operations, respectively. Otherwise it’s done after. Default: False (after).

bias (bool) – If set to False, Linear and LayerNorm layers will not learn an additive bias. Default: True.

forward() will use a special optimized implementation described in FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness if all of the following conditions are met:

Either autograd is disabled (using torch.inference_mode or torch.no_grad) or no tensor argument requires_grad

training is disabled (using .eval())

batch_first is True and the input is batched (i.e., src.dim() == 3)

activation is one of: "relu", "gelu", torch.functional.relu, or torch.functional.gelu

at most one of src_mask and src_key_padding_mask is passed

if src is a NestedTensor, neither src_mask nor src_key_padding_mask is passed

the two LayerNorm instances have a consistent eps value (this will naturally be the case unless the caller has manually modified one without modifying the other)

If the optimized implementation is in use, a NestedTensor can be passed for src to represent padding more efficiently than using a padding mask. In this case, a NestedTensor will be returned, and an additional speedup proportional to the fraction of the input that is padding can be expected.

Pass the input through the encoder layer.

src (Tensor) – the sequence to the encoder layer (required).

src_mask (Optional[Tensor]) – the mask for the src sequence (optional).

src_key_padding_mask (Optional[Tensor]) – the mask for the src keys per batch (optional).

is_causal (bool) – If specified, applies a causal mask as src mask. Default: False. Warning: is_causal provides a hint that src_mask is the causal mask. Providing incorrect hints can result in incorrect execution, including forward and backward compatibility.

see the docs in Transformer.
